

Kraków, 2026

```
import subprocess, sys, os, ssl, certifi, re
import pandas as pd
from Bio import Entrez, Medline
from src.custom_scripts import csv, markdown_to_html, df_to_markdown_table, static_badge, re
from src import search, search_strategy, deduplication
import src
```

Research question

The proposed review answers the question of whether graft choice plays a factor in determining outcomes after primary anterior cruciate ligament reconstruction surgery. And if so, which outcomes are affected by which grafts, as well as the degree of effect specific graft have on each outcome(s) that is/are found to have statistically significant differences in effect sizes. Finally, it answers whether controlling for gender, age and follow-up duration have any effect on the results of the study

Rationale

The current state of knowledge in literature is that there are gold standard surgical techniques for primary anterior cruciate ligament reconstruction surgery in terms of specific grafts and suspension/fixation techniques and tools; however, as advancements in the latter over recent years has opened the door for all soft-tissue and even allogenic grafts as just as viable—if not better—alternative graft choices.

Specifically, the bone-patellar tendon-bone and hamstring tendon autologous grafts have been tried-and-tested and consensus had deemed these two grafts as the 'gold standard' choice for orthopedic surgeons. However, the quadriceps tendon—particularly without the bony attachment—was found to produce non-inferior results compared to those with the patellar bone attachment. This finding opens the door for comparing all soft-tissue grafts with other that share this similar characteristic, such as the hamstring and peroneus longus tendon autologous grafts. As there are numerous studies comparing the various grafts, but not one comprehensive source from which to guide evidence-based practice, this review paper seeks to add to fill this gap in current knowledge by comparing the available randomized controlled trials to produce an evidence level I network meta-analysis to guide clinicians in their practice.

Objectives

This is an important question because total knee arthroplasty (TKA), or total knee replacement is one of the most commonly performed surgical operations in the world. However, studies have found a sole indication for such a procedure, and that is osteoarthritic knee pain. Pain that is chronic, and strong in intensity that it permanently degrades the indicated patients' quality of lives. TKA, however, also limits patients' locomotive capabilities, and is not a procedure that is synonymous with restorative.

The most commonly cited research topic in orthopedic surgery by orthopedics surgeons was anterior cruciate ligament reconstruction (ACLR) surgery, with the link between this and TKA being that the former (ACLR) is the most common cause of non-degenerative and thus, the most common modifiable risk factor for preventing TKA and osteoarthritis. If we are able to answer certain questions through research, then we can advance our collective body of knowledge surrounding one of the most debilitating musculoskeletal conditions and preventing those from developing osteoarthritis after ACLR.

Eligibility criteria

Category	Description	Inclusion/Exclusion criteria	
		Inclusion criteria	Exclusion criteria
P	Adult patients with post-traumatic grade III (complete tear) of unilateral anterior cruciate ligament are included. Both genders are included, and upper limit in age is unrestricted. Only studies with mean age greater than or equal to 18 years old across all patients will be presumed to have targeted a population that is considered skeletally-mature.	Adult patients	Pediatric patients
I	Primary anterior cruciate ligament reconstruction (ACLR) surgeries are included. ACLRs performed simultaneously on both knees, in conjunction with other ligamental reconstructions aside from meniscal, and revision ACLRs were excluded.	Primary ACLR	Revision ACLR or ACL repair (BEAR)

Category	Description	Inclusion/Exclusion criteria	
		Inclusion criteria	Exclusion criteria
C	Autologous grafts: bone-patellar tendon-bone, hamstring tendon, quadriceps tendon, and peroneus longus tendon are included. Allogenic grafts: Achilles tendon and tibialis anterior and posterior are also included. All other graft types, such as synthetic grafts and allogenic versions of commonly used autologous ones are excluded (the Achilles and tibialis tendon grafts are included despite being allogenic because autologous versions cannot be grafted in patients due to the important roles the muscle from which the tendon is attached plays in normal physiologic biomechanics).	Bone-patellar tendon-bone, hamstring tendon, quadriceps, or peroneus longus tendon autologous grafts, or Achilles tendon or tibialis anterior/posterior tendon allogenic grafts.	All other graft types (LET)

Category	Description	Exclusion	
		Inclusion criteria	Exclusion criteria
O Outcome	Primary outcomes of interest were measures of (1) patient-reported outcome measures (PROMs) of patient satisfaction as surrogate measure for subjective quality of life—answering the question of how much is the patient able to live as close to preinjury state; (2) Objective measures of translational and rotational knee stability; and (3) Adverse events and complications. In (1), the primary (most commonly reported) outcomes were included: IKDC subjective knee form (SKF), Lysholm and Tegner. For (2), instrumental laxity (side-to-side difference in millimeters) of antero-posterior translation by the anterior drawer test, pivot shift and Lachman tests were included. For (3) graft failure, including re-re-rupture of graft (confirmed via imaging), clinical failure (defined by clinical failure during objective measures) and revision surgeries (arthroscopic confirmation of tears). Other outcomes that were excluded were due to their deemed incompatibility for comparisons across all grafts. For example, hamstring and quadriceps muscle strength recovery is not a relevant outcome measure for determining effectiveness of ACLR when comparing across grafts in which some were not taken from the patients' hamstring or quadriceps muscle, or is a component that disrupts such biomechanical physiology. Donor site morbidity is another example of an outcome that may be out of range in scope, due to the variety of definitions for consistent applicability in comparisons. Other outcomes such as Marx, KOOS, VAS, were excluded due to the low number of studies that reported these outcome measures, in comparison to the other ones that surrogatively measure the same thing.	Primary outcomes of interest: IKDC subjective, Lysholm, Tegner, pivot shift, Lachman, instrumental laxity, graft rupture, graft failure, revision rate	Secondary outcomes of interest: IKDC, Tegner, pivot shift, Lachman, instrumental laxity, graft rupture, graft failure, revision rate

Category	Description	Inclusion criteria	Exclusion criteria
S Study	Only randomized controlled trials will be included into the De-qualitative and quantitative portions of the review. sign	Randomized controlled trials	Non-randomized clinical trials and observational studies

Search strategies were developed for randomized controlled trials: `pm_bptb.txt`, `pm_ht.txt`, `pm_qt.txt`, `pm_plt.txt`, `pm_at.txt` and `pm_ta.txt` corresponding to PubMed search strategies for patellar, hamstring, quadriceps, peroneus longus, achilles and tibialis anterior and posterior tendones, respectively. From the protocol that was developed, extract key terms from the eligibility criteria for inclusion and exclusion of studies in order to develop a *search strategy*.

The search strategies were 'translated' via regular expressions from PubMed syntax to Embase and Web of Science syntax, store them into the global environment, and save them as plain text files for importing and use as queries for search.

```

population = f""(("pediatric"[tiab] OR "paediatric"[tiab] OR "adolescent"[tia) OR ("revision
acl = f""("anterior cruciate ligament"[mh] OR "anterior cruciate ligament"[tiab] OR "anteri
rct = f""("randomized controlled trial"[pt] OR "randomized controlled trial"[tiab] OR "rand
reviews = f""("review"[pt] OR "review"[tiab] OR "systematic review"[pt] OR "systematic rev
study_design = " NOT ".join([rct, reviews])
outcomes = f""("ikdc"[tiab] OR "lysholm"[tiab] OR "tegner"[tiab] OR (("instrumental laxity"

bptb = f""("bone-patellar tendon-bone"[tiab] OR "bone patellar tendon bone"[tiab] OR "pate
ht = f""("hamstring"[tiab] OR "hamstring tendon"[tiab] OR "semitendinosus"[tiab] OR "gracil
qt = f""("quadriceps"[tiab] OR "quadriceps tendon"[tiab] OR "qt"[tiab]))""
plt = f""("peroneus"[tiab] OR "peroneus longus"[tiab] OR "fibularis longus"[tiab]))""
at = f""("achilles"[tiab] OR "achilles tendon"[tiab]))""
ta = f""("tibialis"[tiab] OR "tibialis anterior"[tiab] OR "tibialis posterior"[tiab]))""

```

```

comparators = f"""{bptb} OR {ht} OR {qt} OR {plt} OR {at} OR {ta}""

df = pd.DataFrame({
    " ": ["P", "I", "C", "O", "S"], # "", "", ""],
    "Categories": ["Population", "Intervention", "Comparators", "Outcomes", "Study Design"],
    "Patellar": [f"NOT {population}", f"{acl}", f"{bptb}", f"{outcomes}", f"{study_design}"],
    "Hamstring": [f"NOT {population}", f"{acl}", f"{ht}", f"{outcomes}", f"{study_design}"],
    "Quadriceps": [f"NOT {population}", f"{acl}", f"{qt}", f"{outcomes}", f"{study_design}"],
    "Peroneus": [f"NOT {population}", f"{acl}", f"{plt}", f"{outcomes}", f"{study_design}"],
    "Achilles": [f"NOT {population}", f"{acl}", f"{at}", f"{outcomes}", f"{study_design}"],
    "Tibialis": [f"NOT {population}", f"{acl}", f"{ta}", f"{outcomes}", f"{study_design}"],
})

df.to_csv("./data/search_strategies_1.csv", encoding = "utf-8")
from IPython.display import Markdown, display
display(Markdown(df.to_markdown(index = False)))

<IPython.core.display.Markdown object>

IC = f"""({acl}) AND ({comparators})""
ICS = f"""({acl}) AND ({comparators}) AND ({study_design})""
print(ICS)

(("anterior cruciate ligament"[mh] OR "anterior cruciate ligament"[tiab] OR "anterior cruciate
subgroups = {"bptb": bptb,
             "ht": ht,
             "qt": qt,
             "plt": plt,
             "at": at,
             "ta": ta}

for x, y in subgroups.items():
    query = f"{acl} AND {y} AND {rct}"
    globals()[f"pm_{x}"] = query
    with open(f"./data/pm_{x}.txt", 'w') as f:
        f.write(query)
    print(query)

pm_queries = {
    "bptb": pm_bptb,
    "ht": pm_ht,
    "qt": pm_qt,
    "plt": pm_plt,
    "at": pm_at,
    "ta": pm_ta
}

```

[illegible][illegible][illegible]

```
for x, y in pm_queries.items():
    query = re.sub(r"\[(.*?)\]", ":", y)
```


(pm_bptb.xml) and parses this into CSV files (pm_bptb.csv).

```
from src import search

---
config:
  theme: light
  curve: stepBefore
---

flowchart TD

A1["pm_bptb"]
B1["pm_ht"]
C1["pm_qt"]
D1["pm_plt"]
E1["pm_at"]
F1["pm_ta"]

A2["em_bptb"]
B2["em_ht"]
C2["em_qt"]
D2["em_plt"]
E2["em_at"]
F2["em_ta"]

A3["wos_bptb"]
B3["wos_ht"]
C3["wos_qt"]
D3["wos_plt"]
E3["wos_at"]
F3["wos_ta"]

A4["co_bptb"]
B4["co_ht"]
C4["co_qt"]
D4["co_plt"]
E4["co_at"]
F4["co_ta"]

G["pubmed"]
H["embase"]
I["wos"]
J["cochrane"]

A1 & B1 & C1 & D1 & E1 & F1 --> G
A2 & B2 & C2 & D2 & E2 & F2 --> H
```

```
A3 & B3 & C3 & D3 & E3 & F3 --> I
A4 & B4 & C4 & D4 & E4 & F4 --> J
```

```
G & H & I & J --> K["records"]
```

```
click A1 "https://raw.githubusercontent.com/d-wkim/d-wkim/refs/heads/main/data//pm_bptb.csv"
click B1 "https://raw.githubusercontent.com/d-wkim/d-wkim/refs/heads/main/data//pm_ht.csv"
click C1 "https://raw.githubusercontent.com/d-wkim/d-wkim/refs/heads/main/data//pm_qt.csv"
click D1 "https://raw.githubusercontent.com/d-wkim/d-wkim/refs/heads/main/data//pm_plt.csv"
click E1 "https://raw.githubusercontent.com/d-wkim/d-wkim/refs/heads/main/data//pm_at.csv"
click F1 "https://raw.githubusercontent.com/d-wkim/d-wkim/refs/heads/main/data//pm_ta.csv"
click A2 "https://raw.githubusercontent.com/d-wkim/d-wkim/refs/heads/main/data//em_bptb.csv"
click B2 "https://raw.githubusercontent.com/d-wkim/d-wkim/refs/heads/main/data//em_ht.csv"
click C2 "https://raw.githubusercontent.com/d-wkim/d-wkim/refs/heads/main/data//em_qt.csv"
click D2 "https://raw.githubusercontent.com/d-wkim/d-wkim/refs/heads/main/data//em_plt.csv"
click E2 "https://raw.githubusercontent.com/d-wkim/d-wkim/refs/heads/main/data//em_at.csv"
click F2 "https://raw.githubusercontent.com/d-wkim/d-wkim/refs/heads/main/data//em_ta.csv"
click A3 "https://raw.githubusercontent.com/d-wkim/d-wkim/refs/heads/main/data//wos_bptb.csv"
click B3 "https://raw.githubusercontent.com/d-wkim/d-wkim/refs/heads/main/data//wos_ht.csv"
click C3 "https://raw.githubusercontent.com/d-wkim/d-wkim/refs/heads/main/data//wos_qt.csv"
click D3 "https://raw.githubusercontent.com/d-wkim/d-wkim/refs/heads/main/data//wos_plt.csv"
click E3 "https://raw.githubusercontent.com/d-wkim/d-wkim/refs/heads/main/data//wos_at.csv"
click F3 "https://raw.githubusercontent.com/d-wkim/d-wkim/refs/heads/main/data//wos_ta.csv"
```

PubMed/MEDLINE

```
pubmed = {
    "pm_bptb": pm_bptb,
    "pm_ht": pm_ht,
    "pm_qt": pm_qt,
    "pm_plt": pm_plt,
    "pm_at": pm_at,
    "pm_ta": pm_ta
}

for x, y in pubmed.items():
    df = search(y)
    globals()[f"{x}"] = df

dfs = [
    "pm_bptb",
    "pm_ht",
    "pm_qt",
    "pm_plt",
    "pm_at",
    "pm_ta",
    "em_bptb",
```

```

    "em_ht",
    "em_qt",
    "em_plt",
    "em_at",
    "em_ta",
    "wos_bptb",
    "wos_ht",
    "wos_qt",
    "wos_plt",
    "wos_at",
    "wos_ta",
    "co_bptb",
    "co_ht",
    "co_qt",
    "co_plt",
    "co_ta",
]

url = "https://raw.githubusercontent.com/d-wkim/d-wkim/refs/heads/main/data/"

for filename in dfs:
    df = pd.read_csv(f"{url}/{filename}.csv", encoding = "utf-8")
    globals()[f"{filename}"] = df

```

Define the names and data frames of the raw datasets

```

pubmed = {
    "pm_bptb": pm_bptb,
    "pm_ht": pm_ht,
    "pm_qt": pm_qt,
    "pm_plt": pm_plt,
    "pm_at": pm_at,
    "pm_ta": pm_ta
}

embase = {
    "em_bptb": em_bptb,
    "em_ht": em_ht,
    "em_qt": em_qt,
    "em_plt": em_plt,
    "em_at": em_at,
    "em_ta": em_ta
}

web_of_science = {
    "wos_bptb": wos_bptb,
    "wos_ht": wos_ht,

```

```

    "wos_qt": wos_qt,
    "wos_plt": wos_plt,
    "wos_at": wos_at,
    "wos_ta": wos_ta
}

```

```

cochrane = {
    "co_bptb": co_bptb,
    "co_ht": co_ht,
    "co_qt": co_qt,
    "co_plt": co_plt,
    "co_ta": co_ta
}

```

PubMed/MEDLINE: Rename columns, clean and create new data frames from the raw datasets.

```

for name, df in pubmed.items():
    df.rename(columns = {
        "id" : "id",
        "pmid" : "pmid",
        "PubDate" : "date",
        "EPubDate" : "epubdate",
        "Source" : "journal_abbr",
        "AuthorList" : "authors",
        "LastAuthor" : "last_author",
        "Title" : "title",
        "Volume" : "volume",
        "Issue" : "issue",
        "Pages" : "pages",
        "LangList" : "language",
        "NlmUniqueID" : "nlmuniqueid",
        "ISSN" : "issn",
        "ESSN" : "essn",
        "PubTypeList" : "study_design",
        "RecordStatus" : "recordstatus",
        "PubStatus" : "pubstatus",
        "ArticleIds" : "articleids",
        "DOI" : "doi",
        "History" : "history",
        "References" : "references",
        "HasAbstract" : "hasabstract",
        "PmcRefCount" : "pmcrefcount",
        "FullJournalName" : "journal",
        "ELocationID" : "elocationid",
        "SO" : "so",
        "abstract" : "abstract",
    })

```

```

        "source" : "source",
        "subgroup" : "subgroup"
    }, inplace = True)

df["id"] = range(1, len(df)+1)

df = pd.DataFrame({
    "id": df["id"],
    "pmid": df["pmid"],
    "source": "pubmed",
    "doi": df["doi"],
    "authors": df["authors"],
    "journal": df["journal"],
    "title": df["title"],
    "abstract": df["abstract"],
    "year": df["year"],
    "language": df["language"]
})

globals()[name] = df
df.to_csv(f"./data/{name}.csv", encoding = "utf-8")
pubmed = pd.concat([pm_bptb, pm_ht, pm_qt, pm_plt, pm_at, pm_ta])
pubmed.to_csv(f"./data/pubmed.csv", encoding = "utf-8")

```

Embase: Rename columns, clean and create new data frames from the raw datasets.

```

for name, df in embase.items():
    df.rename(columns = {
        "Title" : "title",
        "Original Title" : "original_title",
        "Author Names" : "authors",
        "Author Addresses" : "author_addresses",
        "Correspondence Address" : "correspondence_address",
        "Editors" : "editors",
        "AiP/IP Entry Date" : "aip/ip_entry_date",
        "Full Record Entry Date" : "full_record_entry_date",
        "Source" : "journal_full",
        "Source title" : "journal",
        "Publication Year" : "year",
        "Volume" : "volume",
        "Issue" : "issue",
        "First Page" : "first_page",
        "Last Page" : "last_page",
        "Date of Publication" : "date",
        "Publication Type" : "study_design",
        "Conference Name" : "conference_name",
    })

```

```

        "Conference Location" : "conference_location",
        "Conference Date" : "conference_date",
        "Conference Editors" : "conference_editors",
        "ISSN" : "issn",
        "ISBN" : "isbn",
        "Name" : "name",
        "Location" : "location",
        "Date" : "date",
        "Editors" : "editors",
        "Book Publisher" : "book_publisher",
        "Abstract" : "abstract",
        "Original Abstract" : "original_abstract",
        "Author Keywords" : "author_keywords",
        "Entree Drug Index Terms (Major Focus)" : "entree_drug_index_terms_(major_focus)",
        "Entree Drug Index Terms" : "entree_drug_index_terms",
        "Entree Medical Index Terms (Major Focus)" : "entree_medical_index_terms_(major_focus)",
        "Entree Medical Index Terms" : "entree_medical_index_terms",
        "Drug Tradenames" : "drug_tradenames",
        "Drug Manufacturer" : "drug_manufacturer",
        "Device Tradenames" : "device_tradenames",
        "Device Manufacturer" : "device_manufacturer",
        "CAS Registry Numbers" : "cas_registry_numbers",
        "Molecular Sequence Numbers" : "molecular_sequence_numbers",
        "Embase Classification" : "embase_classification",
        "Clinical Trial Numbers" : "clinical_trial_numbers",
        "Article Language" : "language",
        "Summary Language" : "summary_language",
        "Embase Accession ID" : "embase_accession_id",
        "Medline PMID" : "pmid",
        "PUI" : "pui",
        "DOI" : "doi",
        "Full Text Link" : "full_text_link",
        "Embase Link" : "embase_link",
        "Open URL Link" : "open_url_link",
        "Copyright" : "copyright",
        "source" : "source",
        "subgroup" : "subgroup"
    }, inplace = True)

    df["id"] = range(1, len(df)+1)

    df = pd.DataFrame({
        "id": df["id"],
        "pmid": df["pmid"],
        "doi": df["doi"],
        "source": "embase",

```

```

        "authors": df["authors"],
        "journal": df["journal"],
        "title": df["title"],
        "abstract": df["abstract"],
        "year": df["year"],
        "language": df["language"]
    })

    df['pmid'] = df['pmid'].astype(str)
    df["authors"] = df["authors"].str.replace(r"\\'\\[\\]", "", regex = True)
    df["language"] = df["language"].str.replace(r"\\'\\[\\]", "", regex = True)

    globals()[name] = df

    df.to_csv(f"./data/{name}.csv", encoding = "utf-8")
    embase = pd.concat([em_bptb, em_ht, em_qt, em_plt, em_at, em_ta])
    embase.to_csv(f"./data/embase.csv", encoding = "utf-8")

```

Web of Science: Rename columns, clean and create new data frames from the raw datasets.

```

for name, df in web_of_science.items():
    df.rename(columns = {
        "id": "id",
        "i»_PT" : "study_design",
        "AU" : "authors",
        "BA" : "book_authors",
        "BE" : "book_editors",
        "GP" : "book_group_authors",
        "AF" : "authors_full",
        "BF" : "book_author_full_names",
        "CA" : "group_authors",
        "TI" : "title",
        "SO" : "journal",
        "SE" : "book_series_title",
        "BS" : "book_series_subtitle",
        "LA" : "language",
        "DT" : "document_type",
        "CT" : "conference_title",
        "CY" : "conference_date",
        "CL" : "conference_location",
        "SP" : "conference_sponsor",
        "HO" : "conference_host",
        "DE" : "author_keywords",
        "ID" : "keywords_plus",
        "AB" : "abstract",
        "C1" : "addresses",
    })

```

"C3" : "affiliations",
 "RP" : "reprint_addresses",
 "EM" : "email_addresses",
 "RI" : "researcher_ids",
 "OI" : "orcid",
 "FU" : "funding_orgs",
 "FP" : "funding_name_preferred",
 "FX" : "funding_text",
 "CR" : "cited_references",
 "NR" : "cited_reference_count",
 "TC" : "times_cited, wos_core",
 "Z9" : "times_cited, all_databases",
 "U1" : "180_day_usage_count",
 "U2" : "since_2013_usage_count",
 "PU" : "publisher",
 "PI" : "publisher_city",
 "PA" : "publisher_address",
 "SN" : "issn",
 "EI" : "eissn",
 "BN" : "isbn",
 "J9" : "journal_9",
 "JI" : "journal_abbr",
 "PD" : "date",
 "PY" : "year",
 "VL" : "volume",
 "IS" : "issue",
 "PN" : "part_number",
 "SU" : "supplement",
 "SI" : "special_issue",
 "MA" : "meeting_abstract",
 "BP" : "start_page",
 "EP" : "end_page",
 "AR" : "article_number",
 "DI" : "doi",
 "DL" : "doi_link",
 "D2" : "book_doi",
 "EA" : "early_access_date",
 "PG" : "number_of_pages",
 "WC" : "wos_categories",
 "WE" : "web_of_science_index",
 "SC" : "research_areas",
 "GA" : "ids_number",
 "PM" : "pmid",
 "OA" : "open_access_designations",
 "HC" : "highly_cited_status",
 "HP" : "hot_paper_status",


```

        "DA" : "date_of_export",
        "UT" : "ut (unique_wos_id)",
        "source" : "source",
        "subgroup": "subgroup"
    }, inplace = True)

df["id"] = range(1, len(df)+1)

df = pd.DataFrame({
    "id": df["id"],
    "pmid": df["pmid"],
    "source": "web_of_science",
    "doi": df["doi"],
    "authors": df["authors"],
    "journal": df["journal"],
    "title": df["title"],
    "abstract": df["abstract"],
    "year": df["year"],
    "language": df["language"]
})

df['pmid'] = df['pmid'].astype(str)
df["authors"] = df["authors"].str.replace(r"\\\[\\]", "", regex = True)
df["language"] = df["language"].str.replace(r"\\\[\\]", "", regex = True)
globals()[name] = df

df.to_csv(f"./data/{name}.csv", encoding = "latin-1")
wos = pd.concat([wos_bptb, wos_ht, wos_qt, wos_plt, wos_at, wos_ta])
wos.to_csv(f"./data/wos.csv", encoding = "latin-1")

```

Cochrane Library

```

for name, df in cochrane.items():
    df.rename(columns = {
        "CENTRAL ID": "CENTRAL_ID",
        "Author(s)": "authors",
        "Title": "title",
        "Source": "source",
        "Year": "year",
        "Abstract": "abstract",
        "PubMed ID": "pmid",
        "DOI": "doi",
        "URL": "url",
    }, inplace = True)

df["id"] = range(1, len(df)+1)

```

```

df = pd.DataFrame({
    "id": df["id"],
    "pmid": df["pmid"],
    "source": "Cochrane",
    "doi": df["doi"],
    "authors": df["authors"],
    "journal": df["source"],
    "title": df["title"],
    "abstract": df["abstract"],
    "year": df["year"],
    "language": "English"
})

df['pmid'] = df['pmid'].astype(str)
df["authors"] = df["authors"].str.replace(r"\\'\\[\\]", "", regex = True)
df["language"] = df["language"].str.replace(r"\\'\\[\\]", "", regex = True)
globals()[name] = df
df.to_csv(f"./data/{name}.csv", encoding = "utf-8")
cochrane = pd.concat([co_bptb, co_ht, co_qt, co_plt, co_ta])
cochrane.to_csv(f"./data/cochrane.csv", encoding = "utf-8")

```

Define variables by grouping the subgroup data frames.

```

patellar = [pm_bptb, em_bptb, wos_bptb]
hamstring = [pm_ht, em_ht, wos_ht]
quadriceps = [pm_qt, em_qt, wos_qt]
peroneus = [pm_plt, em_plt, wos_plt]
achilles = [pm_at, em_at, wos_at]
tibialis = [pm_ta, em_ta, wos_ta]

```

Combine dataframes by database across subgroups and write them to CSV

```

bptb = pd.concat(patellar).to_csv(f"./data/patellar.csv", encoding = "utf-8")
ht = pd.concat(hamstring).to_csv(f"./data/hamstring.csv", encoding = "utf-8")
ta = pd.concat(quadriceps).to_csv(f"./data/tibialis.csv", encoding = "utf-8")
at = pd.concat(peroneus).to_csv(f"./data/achilles.csv", encoding = "utf-8")
plt = pd.concat(achilles).to_csv(f"./data/peroneus.csv", encoding = "utf-8")
qt = pd.concat(tibialis).to_csv(f"./data/quadriceps.csv", encoding = "utf-8")

subgroups = [patellar, hamstring, quadriceps, peroneus, achilles, tibialis]

```

Combine dataframes by subgroup across databases and write them to CSV.

```

records = pd.concat([pubmed, embase, wos, cochrane])
records.to_csv(f"./data/records.csv", encoding = "utf-8")

len(records)

```

1704

Deduplication

The 'gold-standard' or consensus agreement among researchers seems to converge on the idea that removal of duplicate records is best performed in a process that involves three ordered stages. The first is deduplication based on a unique record identifier, such as a digital object identifier (DOI) number, PubMed identifier (PMID) number, or clinicaltrials.gov (NCT) number. Then, as according to as described for the second stage, the remaining records were deduplicated based on a concatenated column consisting of the title, author, and year. The title was standardized by converting to sentence case, and punctuation marks and white spaces were removed. The character length was decreased to 50. For the authors column, the last name of the first author was chosen to be used. For the year of publication, the year was extracted from the date of publication in the electronic version of the journal and converted into a string data structure.

input file(s): records.csv, output file(s): doi_deduplicated.csv, pmid_deduplicated.csv, title+author+year_deduplicated.csv, and title+year_deduplicated.csv.

```
def deduplication(df, columns):
    if isinstance(columns, str):
        columns = [columns]

    nulls_mask = df[columns].isnull().any(axis=1)

    df_nulls = df[nulls_mask]
    df_non_nulls = df[~nulls_mask]

    duplicates_mask = df_non_nulls.duplicated(
        subset=columns,
        keep=False,
    )

    df_duplicates = df_non_nulls[duplicates_mask]
    df_non_duplicates = df_non_nulls[~duplicates_mask]

    df_kept = df_duplicates.drop_duplicates(
        subset=columns,
        keep="first",
    )

    df_removed = df_duplicates.loc[
        ~df_duplicates.index.isin(df_kept.index)
    ]

    df_unique = df_non_nulls.drop_duplicates(subset=columns)

    df_deduplicated = pd.concat(
        [df_non_duplicates, df_kept, df_nulls],
```

```

        ignore_index=True,
    )

    return df_deduplicated
records = deduplication(records, "doi")
len(records)
925
df = deduplication(records, "pmid")
df.to_csv("./data/title_screening.csv", encoding = "utf-8")
len(df)
908

```

```

import pandas as pd

df = pd.read_csv("./data/full_text_screening.csv", encoding = "utf-8")
df = df[["DOI", "Author", "Publication Year"]]
df = df.sort_values(by = ["Publication Year"], ascending = False)
df = df.sort_values(by = ["Author"], ascending = True)

import pandas as pd

df = pd.read_csv("./data/screening.csv", encoding = "utf-8")

included = df[df["screening"] == "Included"]
excluded = df[df["screening"] == "Excluded"]
df = included.drop(columns = ["screening"])
df.info()

<class 'pandas.DataFrame'>
Index: 68 entries, 0 to 99
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   id           68 non-null    int64
1   study        68 non-null    str
2   subgroups    68 non-null    str
3   outcomes     68 non-null    str
dtypes: int64(1), str(3)
memory usage: 7.5 KB

md = df.to_markdown()

from IPython.display import display, HTML, Markdown
display(Markdown(md))

```

<IPython.core.display.Markdown object>